

ARQUITECTURA DE SOFTWARE

PRÁCTICO

Alta Disponibilidad vs. Resiliencia

A menudo se confunden, pero son conceptos distintos con objetivos diferentes. Entender la diferencia ayuda a diseñar sistemas robustos.

Alta Disponibilidad

Es hacer todo lo posible para que el sistema **nunca se caiga**. Se logra duplicando equipos (redundancia) para que si uno falla, otro tome el control.

Su métrica principal es el **Uptime**: el porcentaje de tiempo que el sistema está en línea. El objetivo clásico es "cinco nueves" (99.999%).

Resiliencia

Es aceptar que **las cosas se van a romper** y diseñar el sistema para que se recupere rápido sin que el usuario lo note demasiado.

Busca mantener la operación bajo condiciones adversas. No evita los fallos, pero minimiza su impacto y acelera la recuperación.

Escalabilidad y Sobreviviendo al Caos

Cuando nuestra aplicación se vuelve popular (por ejemplo, durante un **Hot Sale**), un solo servidor no da abasto. Aquí entra el concepto de **Elasticidad y Auto Scaling**: el sistema agrega o quita servidores automáticamente según la demanda.



Escalado Vertical

Comprarle más RAM y mejor procesador al servidor actual. Es sencillo, pero tiene un **límite físico**: no puedes comprar una computadora infinita.



Escalado Horizontal

En lugar de una computadora gigante, usamos **10 computadoras pequeñas** que trabajan en equipo. Es más flexible y económico a gran escala.

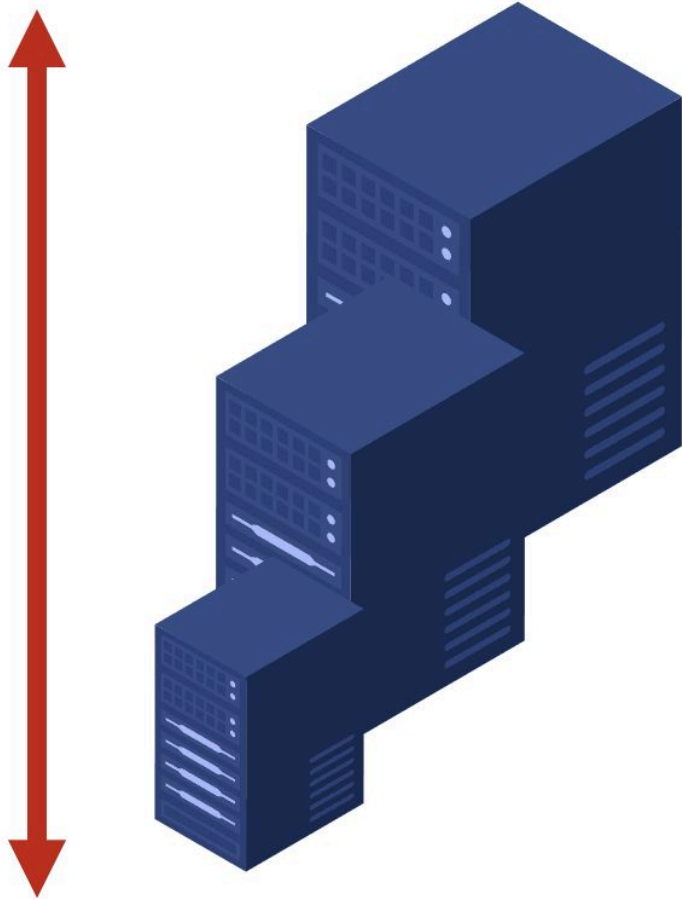


Auto Scaling

El sistema detecta que el CPU llegó al **70%** y crea 5 copias más de la app en Go automáticamente. Cuando el tráfico baja, las destruye para ahorrar dinero.

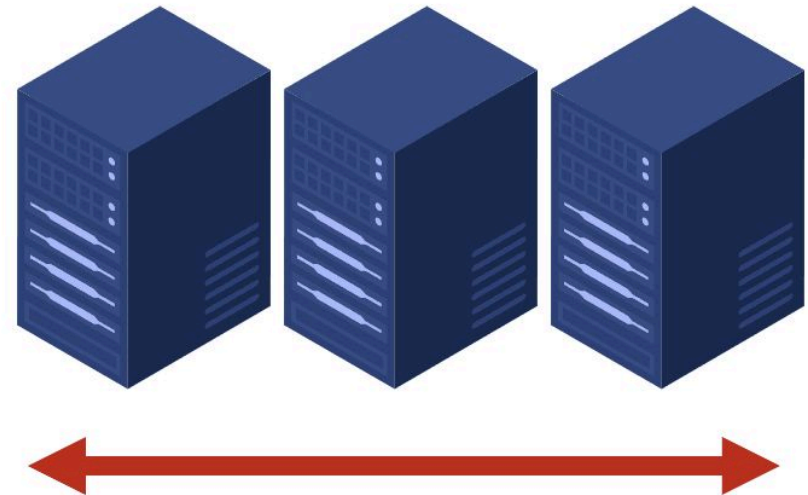
Vertical Scaling

Increase or decrease the capacity of existing services/instances.



Horizontal Scaling

Add more resources like virtual machines to your system to spread out the workload across them.



Patrones de Resiliencia en el Código

Agregar servidores no sirve de nada si nuestro código no sabe manejar los errores de red. Estos tres patrones son simples pero vitales:

1

Retry con Backoff Exponencial

Si la conexión falla, reintentamos. Pero no de inmediato: esperamos **1 segundo**, luego **2**, luego **4**. Así evitamos saturar la red mientras el servicio se recupera.

2

Circuit Breaker (El Interruptor)

Funciona como la llave térmica de tu casa. Si un servicio está caído, el interruptor "salta" y **corta el tráfico** para no colapsar todo el sistema. Periódicamente prueba si el servicio ya se recuperó.

3

Fallback (El Plan B)

Si algo falla, ofrecemos una alternativa en lugar de un error. Si el sistema de **recomendaciones con IA** está caído, mostramos "Los 10 productos más vendidos". El sistema se degradó, pero no falló.

★ EL CORE DE LA CLASE

Balanceadores de Carga

Si tenemos 5 servidores trabajando en equipo, ¿cómo sabe el celular del usuario a cuál conectarse? Aquí entra el **Balanceador de Cargas (Load Balancer)**: un software de altísimo rendimiento que recibe todas las peticiones y las reparte como un "director de tránsito".

HAProxy

Excelente para tráfico puro. Conocido por su **velocidad extrema** y estabilidad en entornos de alta demanda.

Envoy

Muy moderno, diseñado para entornos **gigantes y complejos** en la nube. Es la base de muchos sistemas de microservicios.

NGINX ★

La **navaja suiza** de internet. Funciona como servidor, proxy y balanceador. Es el estándar de la industria y lo usaremos en nuestro laboratorio.

¿Cómo Reparten el Tráfico los Balanceadores?

El balanceador usa fórmulas matemáticas / **algoritmos** para decidir a qué servidor enviar cada petición. Cada uno tiene sus ventajas según el contexto.

1 Round-Robin

Reparte en orden cíclico: servidor 1, luego 2, luego 3, y vuelve al 1. Es justo, pero asume que todos los servidores son igual de potentes.

2 Weighted Round-Robin

Si el servidor 1 es nuevo y potente, le damos más "peso". Recibe, por ejemplo, **3 peticiones por cada 1** que recibe el servidor viejo.

3 Least Connections

Envía al usuario al servidor con **menos conexiones activas** en ese instante. Ideal cuando algunas tareas tardan mucho más que otras.

4 Hashing Consistente

Imagina un reloj circular con los servidores en distintas "horas". Si un servidor se apaga, **solo los usuarios de esa zona** se mueven al siguiente; el resto ni se entera.

Aplicaciones Stateless (Lo conocen... verdad?)

Para que el Auto Scaling y el Balanceador funcionen de verdad, nuestra app en Go debe seguir la regla de oro de la Clase 1 de DSW: **no guardar información del usuario en su memoria RAM local.**

✗ Arquitectura Stateful (Con Estado)

La app guarda la sesión o el carrito en su propia memoria. Esto obliga a usar IP Hash para que el usuario siempre vuelva al mismo servidor. **Si ese servidor se rompe, el usuario pierde su sesión.**

✓ Arquitectura Stateless (Sin Estado)

La aplicación nace en la petición y muere en la respuesta. Cualquier servidor libre puede atender al usuario.

Configurando NGINX

Cuando ponemos NGINX frente a nuestra app de Go, hay configuraciones clave para que sea verdaderamente resiliente y eficiente:



Chequeos de Salud (Health Checks)

NGINX detecta rápido si un servidor de Go "murió" y lo saca de la rotación. Configuramos tolerancias: por ejemplo, **3 errores antes de considerarlo muerto**, y lo dejamos fuera por 30 segundos.



Reutilización de Conexiones (Keepalive)

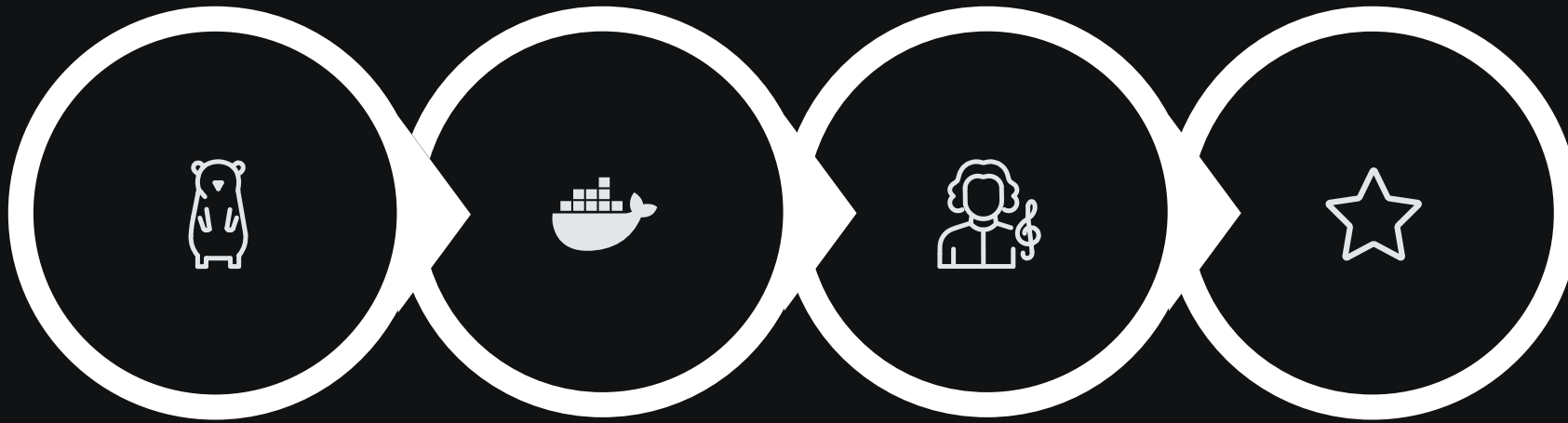
Abrir y cerrar conexiones miles de veces por segundo consume mucha CPU. Usamos **keepalive** para mantener, por ejemplo, 32 conexiones permanentemente abiertas entre NGINX y Go. El sistema se vuelve mucho más rápido.



Errores 502 vs 504

502 Bad Gateway: el servidor de Go explotó o está mal configurado; NGINX no pudo conectarse. **504 Gateway Timeout:** NGINX se conectó, pero la base de datos fue tan lenta que NGINX se cansó de esperar y cortó la llamada.

Tu Propio Balanceador con Go, Docker y NGINX



Crear API Go

**Empaquetar
Docker**

**3 Réplicas
Compose**

**Configurar
NGINX**



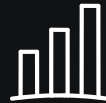
 ¡Sigue la demo en vivo!

Monitoreo (Tema de la Próxima Clase)



Logs (Registros)

El "diario íntimo" del sistema. Cada vez que hay un error, el programa en Go escribe un mensaje detallando **qué pasó y a qué hora**. Fundamental para investigar incidentes.



Métricas

Números y estadísticas en tiempo real. Por ejemplo: "**Estamos usando el 85% de la RAM**" o "**Recibimos 5,000 peticiones por segundo**". Alertan antes de que el sistema colapse.



Trazas (Tracing)

Como una petición pasa por muchos servidores, las trazas dibujan el **camino exacto** que siguió un usuario desde el clic hasta la base de datos, midiendo el tiempo en cada parada.